PROGRAMAÇÃO DISTRIBUÍDA

Exame da Época Normal

10 de janeiro de 2024

Com consulta / Duração: 120 minutos / 12 valores

Considere um sistema distribuído elementar, desenvolvido em Java, que permite que vigilantes de um campus possam acompanhar, em tempo real, a passagem de cartões nos sensores instalados nas diversas portas dos edifícios. Cada sensor e cartão possui um identificador único que o identifica de forma unívoca. Também é possível, através do sistema pretendido, a troca direta de mensagens de texto entre vigilantes.

O sistema é constituído por três aplicações distintas (servidor/*server*, sensor e vigilante/*guard*) com as seguintes funcionalidades e interações:

- A aplicação sensor começa por estabelecer uma ligação TCP com o servidor e enviar-lhe, sob a forma de uma linha de texto (sequência de caracteres ascii terminada com uma mudança de linha), o identificador do sensor a que está associada. Depois de esta fase inicial, transmite o código associado aos cartões que vão sendo detetados;
- A aplicação vigilante, utilizada por cada um dos vigilantes nos seus equipamentos, disponibiliza um serviço remoto RMI que permite que: (1) sejam visualizadas mensagens de texto; (2) seja fornecido um conjunto de referências remotas para serviços RMI de vigilantes; e (3) seja fornecida a informação associada a um cartão que tenha sido detetado por um sensor (identificador do sensor, indentificador do cartão e momento de deteção). Esta aplicação começa por registar o seu serviço remoto no *registry* RMI local e enviar a respetiva referência remota para o servidor, via UDP, sob a forma de um objeto serializado (serialização binária Java). Também aguarda continuamente pela introdução de mensagens de texto, através da entrada standard (`System.in`), que encaminha, de imediato, via RMI, para os restantes vigilantes com referências remotas presentes no último conjunto fornecido;
- O servidor aguarda continuamente, via UDP, pela receção de referências remotas de serviços RMI de vigilantes e mantém um conjunto atualizado destas. Também fornece este conjunto de referências aos vigilantes, através dos respetivos serviços RMI, sempre que existe alguma atualização. Quando recebe, via TCP, uma indicação de cartão detetado por um determinado sensor, o servidor notifica deste facto os vigilantes, via RMI, recorrendo ao conjunto de referências remotas que mantém.

Complete as partes em falta de código apresentado nas páginas seguintes e responda a algumas das questões colocadas. Para o efeito, deve usar duas folhas de prova novas, colocando em cada uma as respostas correspondentes às seguintes alíneas:

- Folha 1 - [1] a [10], [17], [19] e [20];
- Folha 2 - [11] a [16] e [18].

Identifique devidamente as alíneas correspondentes às respostas que apresentar e ignore a questão dos *import* e *packages*.

/* [1] Objetos deste tipo devem poder ser passados como argumentos em chamadas RMI. Introduza eventuais alterações para que este requisito possa ser satisfeito. */

```
public class SensorInfo {
    private String sensorId;
    private String cardId;
    private Date date;

    public SensorInfo(String sensorId, String cardId, Date date) {
        this.sensorId = sensorId;
        this.cardId = cardId;
        this.date = date;
    }

     //Assuma que os “getters” e “setters”, bem como os métodos “equals”, “hashCode” e
     //“toString” de esta classe já estão definidos. Não os implemente.
     ...
}
```

/* Esta aplicação elementar simula o funcionamento de um sensor e possui apenas caráter informativo. Não é para completar nem alterar. */

```
public class Sensor {
    public static void main(String [] args) throws IOException {
        if(args.length < 2) {
            System.out.println("Two arguments required in the command-line: <server address>
                        and <tcp listening port>");
            return;
        }

        int senorId =(int)(Math.random()*1000) + 1000;

        try( Socket s = new Socket(args[0], Integer.parseInt(args[1]) );
            PrintWriter pw = new PrintWriter(s.getOutputStream(), true)) {
```

```
            pw.println("Sensor_" + senorId);

            while(true) {
                try{
                    long time = (long)(Math.random()*1000) + 5000;
                    Thread.sleep(time);
                }catch (InterruptedException ex){continue;}

                int cardId =(int)(Math.random()*1000) + 1000;
                pw.println("Card_" + cardId);
            }
        }
    }
}
```

```
/* Interface remota implementada pelo serviço RMI disponibilizado pela aplicação vigilante. */

public interface GuardInterface extends Remote{
    String getName() throws RemoteException;
    void setMsg(String msg) throws RemoteException;
    void setGuards(Set<Remote> buildingGuardsSet) throws RemoteException;
    void notifyCardDetected(SensorInfo info) throws RemoteException;
}
```

```
public class Server {

    public static final int UDP_PORT = 6001;
    public static final int TCP_PORT = 6001;
    public static final int MAX_LENGTH = 10000;

    private Set<Remote> guardsSet = new HashSet<>();

    private class UdpThread extends Thread {
```

```
@Override
public void run() {

  /* [2] Crie o socket UDP "ds" associado ao porto UDP_PORT */
    try (... ds = ...){

        while(true){

            /* [3] Aguarde pela receção de um datagrama em "ds" */
            DatagramPacket pkt = ...
            ...

            /* [4] Crie "oin" para permitir a "deserialização" do conteúdo do
               datagrama recebido e "deserialize-o" para "newRef" */
            try(ObjectInputStream oin = ...) {

                Remote newRef = ...

                /* [5] Garanta que a execução do bloco seguinte, em que se acede ao
                   conjunto "guardsSet", é feita de forma mutuamente exclusiva */
                ... { //block start

                    if (!guardsSet.add(newRef))
                        continue; //Remote reference already exists

                    boolean changed;
                    List<Remote> guardsToRemove = new ArrayList<>();

                    do {
                        changed = false;
                        guardsToRemove.clear();

                        /* [6] Forneça, através do respetivo serviço RMI, o conjunto
                           "guardsSet" a todos os vigilantes com referências remotas
                           presentes em "guardsSet". Faça um uso apropriado de
                           "guardsToRemove" */
                        ...
```

```
                        for (Remote r : guardsToRemove) {
                            if(guardsSet.remove(r)) {
                                changed = true;
                            }
                        }

                    } while(changed);
                } //block end

            }catch (IOException | ClassNotFoundException ex) {
                System.out.println(ex);
            }
        } //while
    } catch (IOException ex) {
        System.out.println(ex);
    }
  }
}

private class TcpAcceptThread implements Runnable {
    @Override
    public void run() {
      /* [7] Crie um socket à escuta no porto TCP "TCP_PORT" e entre num ciclo em que vai
          aceitando ligações. Cada uma destas ligações deve ser atendida por uma thread "t"
          baseada na classe "TcpProcessThread" e a correr em modo "daemon" */

      try(... serverSocket = ...){
          while(true) {
              ... s = ...
              ... t = ...
              ...                 }
      } catch (IOException ignored) {}
    }
}

private class TcpProcessThread implements Runnable {
    private Socket s;
```

```
public TcpProcessThread(Socket s) { this.s = s; }


@Override
public void run() {
  /* [8] Crie um "BufferedReader" que permita aguardar pela receção de linhas de texto
     em "s" */
   try(BufferedReader bin = ...) {
        String sensorId = bin.readLine(); //First text line is the sensor's identifier


        String cardId; //Next lines are identifiers of detected cards
        while((cardId = bin.readLine())!=null) {
            notifyGuards(new SensorInfo(sensorId, cardId, new Date()));
        }
    }catch (IOException ex){
        System.out.println(ex);
    }finally {
        try { s.close(); } catch (IOException ignored) {}
    }
}


private void notifyGuards(SensorInfo info) {
    List<Remote> toRemove = new ArrayList<>();


    /* [9] Garanta que a execução do bloco seguinte, em que se acede ao conjunto
       "guardsSet", é feita de forma mutuamente exclusiva */
    ... { //block start


        /* O seguinte pedaço de código em falta destina-se a fornecer, através do
           respetivo serviço RMI, a informação representada por "info" a todos os
           vigilantes com referências remotas presentes em "guardsSet". Devido à sua
           semelhança com a alínea [6], este pedaço de código não é para ser
           completado. */

       ...


        for (Remote r : toRemove) {
            guardsSet.remove(r);
        }
```

```
            } //block end
        }
    }

    public void start() {
       /* [10] Lance, em modo “daemon”, uma thread baseada na classe UdpThread e outra na
          classe   TcpAcceptThread */

       ... udpThread = ...
       ...

       ... tcpThread = ...
       ...

       System.out.println("Press <Enter> to exit");
       try { System.in.read(); } catch (IOException ignored) {}
    }

    public static void main(String[] args) throws IOException {
        new Server().start();
    }
}
```

/* [11] Complete a declaração da classe “GuardService” para que seja um serviço RMI que ofereça a interface remota “GuardInterface” */

```
public class GuardService {

    private static final String EXIT = "exit";
    private String name;
    private Set<Remote> guardsSet = null;

    /* [12] Defina um construtor que receba como argumento “String name” */
    ...

    @Override
    public String getName() throws RemoteException{
```

```
    return name;
}

@Override
public void setMsg(String msg) throws RemoteException {
    System.out.println(msg);
}

@Override
public void notifyCardDetected(SensorInfo info) throws RemoteException {
    System.out.println("Card detected: " + info);
}

/* [13] Complete a declaração dos métodos “setGuards” e “notifyOtherGuards” para garantir
   que, em cada instante, apenas uma thread pode executar um deles sobre o mesmo objeto. */

@Override
public void setGuards(Set<Remote> buildingGuardsSet) throws RemoteException {
    this.guardsSet = buildingGuardsSet;
}

public void notifyOtherGuards(String msg, Remote myServiceRef) {
    if(guardsSet == null)
        return;

    List<Remote> toRemove = new ArrayList<>();

    /* [14] Coloque, recorrendo ao respetivo serviço RMI, “msg” em todos os vigilantes com
       referências remotas presentes em “guardsSet”, com exceção da origem (referência
       remota igual a “myServiceRef”). Faça um uso apropriado de “toRemove” */
    ...

    for(Remote r:toRemove) {
        if(guardsSet.remove(r))
            System.out.println("Remote reference removed from guards set: " + r);
    }
}
```

```
public static void main(String [] args) {
    if(args.length<3) {
        System.out.println("Three arguments required in the command-line: <server address>
                            <server udp port> <guard name>!");
        return;
    }

    /* [15] Crie o socket UDP "ds" num porto automático, bem como os objetos "bout" e
       "oout" que serão usados para serialização binária para um array de bytes. */
    try(DatagramSocket ds = ...;
        ByteArrayOutputStream bout = ...;
        ObjectOutputStream oout = ...) {

        String servAddr = args[0];
        int servUdpPort = Integer.parseInt(args[1]);
        String myName = args[2];

        try{
            LocateRegistry.createRegistry(java.rmi.registry.Registry.REGISTRY_PORT);
        }catch (Exception ignored){}

        GuardService myService = new GuardService(myName);

        /* [16] Registe o serviço RMI "myService" no registry local com o nome dado pelo
           valor da string "myName". */
        ...

        Remote myServiceRef = Naming.lookup("rmi://localhost/" + myName);

        /* [17] Explique, de forma sucinta e objetiva, as diferenças e semelhanças
           principais existentes entre o objeto "myService", que consiste no serviço RMI
           lançado, e o objeto "myServiceRef", também designado de proxy ou stub para o
           serviço. */

        /* [18] Envie "myServiceRef" para o servidor, via "ds", sob a forma de um objeto
           serializado. Recorra aos objetos já criados "ds", "oout" e "bout". */
        ...
        oout.flush();
```

```
            ... pkt = ...
            ds...

            BufferedReader bin = new BufferedReader(new InputStreamReader(System.in));
            String msg;

            System.out.println("Type '" + EXIT + "' to exit.");

            do{
                msg = bin.readLine();
                myService.notifyOtherGuards(myName + ": " + msg, myServiceRef);
            }while(!msg.equals(EXIT));

            Naming.unbind((“rmi://localhost/” + myName);

            /* [19] Se a seguinte instrução não for invocada, a aplicação não termina apesar do
               final do método “main” ser atingido. Explique a razão. */
            UnicastRemoteObject.unexportObject(myService, true);

        }catch (NotBoundException | IOException ex){
            System.out.println(ex);
        }
    }
}
```

/* [20] Assuma que o servidor também inclui uma API REST elementar, desenvolvida em Spring Boot, cujo controlador está parcialmente representado de seguida. Apresente, para cada um dos quatro “endpoints” definidos (A a D), um exemplo concreto de pedido HTTP. Inclua todos os elementos necessários à exemplificação (verbo HTTP, URI, etc.), à semelhança do seguinte exemplo de hipotética resposta: POST /clients/pt com {“name”:”David”,”nif”:999999} no conteúdo do pedido. */

```
@RestController
public class ServerController {

    @GetMapping("/guards") // (A)
    public List<String> getGuardsNames() {
          /* Não é para completar o código do método */
          ...
    }

     @DeleteMapping("/guards/{name}") // (B)
     public boolean deleteGuard_v1(@PathVariable("name") String guardNameToRemove) {
          /* Não é para completar o código do método */
          ...
     }

     @DeleteMapping("/guards") // (C)
     public boolean deleteGuard_v2 (@RequestParam("name") String guardNameToRemove) {
          /* Não é para completar o código do método */
          ...
     }

     @PostMapping("/detected_cards") // (D)
     public boolean addDetectedCard (@RequestHeader(“Content-Length”) long length,
                                     @RequestBody SensorInfo info) {
          /* Não é para completar o código do método */
          ...
     }
}
```